

# WatchParty

## Technical Report

**S26CS6.401 — Software Engineering — Project 3  
Deliverable**

Domain: Entertainment — OTT / Social Viewing

**Repository:** [github.com/Zenith053/WatchParty](https://github.com/Zenith053/WatchParty)

**Submitted:** 25th April 2026

### Team Members

|          |                  |
|----------|------------------|
| Member 1 | Peeyush Prashant |
| Member 2 | Rahul Saha       |
| Member 3 | Shubham Paliwal  |
| Member 4 | Samir Saurabh    |

# Contents

---

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>Executive Summary</b>                                                            | <b>3</b>  |
| <b>1 Task 1: Requirements and Subsystems</b>                                        | <b>4</b>  |
| 1.1 Functional Requirements . . . . .                                               | 4         |
| 1.2 Non-Functional Requirements . . . . .                                           | 4         |
| 1.3 Architecturally Significant Requirements . . . . .                              | 5         |
| 1.4 Subsystem Overview . . . . .                                                    | 6         |
| <b>2 Task 2: Architecture Framework</b>                                             | <b>8</b>  |
| 2.1 Stakeholder Identification (IEEE 42010) . . . . .                               | 8         |
| 2.1.1 Stakeholders and Concerns . . . . .                                           | 8         |
| 2.1.2 Viewpoints and Views . . . . .                                                | 8         |
| 2.1.3 C1: System Context Diagram . . . . .                                          | 9         |
| 2.1.4 C2: Container Diagram . . . . .                                               | 10        |
| 2.1.5 C3: Component Diagram . . . . .                                               | 11        |
| 2.2 Major Design Decisions — Architecture Decision Records . . . . .                | 11        |
| 2.2.1 ADR-01: WebSocket for Real-Time Communication . . . . .                       | 11        |
| 2.2.2 ADR-02: Host-Authoritative Sync Model . . . . .                               | 12        |
| 2.2.3 ADR-03: Command Pattern for Message Dispatch . . . . .                        | 13        |
| 2.2.4 ADR-04: Strategy Pattern for Database Abstraction . . . . .                   | 14        |
| <b>3 Task 3: Architectural Tactics and Patterns</b>                                 | <b>15</b> |
| 3.1 Architectural Tactics . . . . .                                                 | 15        |
| 3.1.1 Tactic 1: In-Memory Broadcast for Latency (NFR-01) . . . . .                  | 15        |
| 3.1.2 Tactic 2: Graceful Degradation for Fault Tolerance (NFR-02, NFR-03) . . . . . | 15        |
| 3.1.3 Tactic 3: Pre-Upgrade Authentication for Security (NFR-06) . . . . .          | 15        |
| 3.1.4 Tactic 4: Rate Limiting for Scalability (NFR-04) . . . . .                    | 16        |
| 3.1.5 Tactic 5: Module Decomposition for Maintainability (NFR-08) . . . . .         | 16        |
| 3.2 Implementation Patterns . . . . .                                               | 16        |
| 3.2.1 Pattern 1: Command Pattern . . . . .                                          | 17        |
| 3.2.2 Pattern 2: Strategy Pattern (Database Abstraction) . . . . .                  | 17        |
| <b>4 Task 4: Prototype Implementation and Analysis</b>                              | <b>19</b> |
| 4.1 Prototype Development . . . . .                                                 | 19        |
| 4.1.1 End-to-End Functionality Implemented . . . . .                                | 19        |
| 4.1.2 Technology Stack . . . . .                                                    | 19        |
| 4.1.3 Test Results and Coverage . . . . .                                           | 20        |
| 4.2 Architecture Analysis — Pattern Comparison . . . . .                            | 20        |
| 4.2.1 Implemented Architecture: Event-Driven with Mediator . . . . .                | 20        |
| 4.2.2 Comparison Architecture: Layered (N-Tier) Architecture . . . . .              | 20        |
| 4.2.3 Quantified Comparison: NFR-01 (Sync Latency) . . . . .                        | 21        |
| 4.2.4 Quantified Comparison: NFR-03 (Fault Tolerance — Host Migration) . . . . .    | 21        |
| 4.2.5 Trade-Off Summary . . . . .                                                   | 22        |
| <b>5 Reflections and Lessons Learned</b>                                            | <b>23</b> |
| 5.1 Design Smell Refactoring . . . . .                                              | 23        |
| 5.2 NFR Compliance Journey . . . . .                                                | 23        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| 5.3 Architecture Evolution . . . . .                                | 23        |
| <b>6 Individual Contributions</b>                                   | <b>25</b> |
| <b>A Appendix</b>                                                   | <b>26</b> |
| A.1 How to Run . . . . .                                            | 26        |
| A.2 Additional ADRs . . . . .                                       | 26        |
| A.2.1 ADR-05: Dual-Layer State Store (Memory + Redis) . . . . .     | 26        |
| A.2.2 ADR-06: Token-Based Room Security with Pre-Upgrade Validation | 27        |
| A.2.3 ADR-07: In-Memory Sliding-Window Rate Limiter . . . . .       | 27        |
| A.3 Design Pattern Summary . . . . .                                | 27        |
| A.4 Room State Machine . . . . .                                    | 28        |
| A.5 Project Structure . . . . .                                     | 28        |

## Executive Summary

---

WatchParty is a real-time, browser-based social viewing platform that enables distributed friend groups to watch online videos together with synchronized playback, live chat, and a collaborative vote-to-watch queue. Built as a full-stack Node.js application using Express, WebSockets (`ws`), PostgreSQL, and Redis, the system requires no account creation or software installation — users interact entirely through their web browser.

The prototype implements two end-to-end non-trivial user journeys:

1. **Synchronized Viewing** — A user creates a room, shares an invite link, and all participants watch a YouTube video in perfect sync with host-controlled playback (play/pause/seek), late-join catch-up, and automatic host migration on disconnect.
2. **Collaborative Queue** — Any room member can nominate video URLs, upvote entries, and trigger majority-based skip voting. When a video ends, the top-voted entry auto-plays.

The architecture employs 6 design patterns (Command, State Machine, Observer/Pub-Sub, Strategy, Singleton, Facade), is validated by 13 test suites with 85%+ statement coverage, and addresses all 8 non-functional requirements through purpose-built architectural tactics.

# 1 Task 1: Requirements and Subsystems

## 1.1 Functional Requirements

| Priority           | ID    | Requirement         |         | Description                                                                                                                                                  |
|--------------------|-------|---------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Must Have</b>   | FR-01 | Room Creation       |         | Any user can create a watch room and receive a shareable invite link; no account required to join as a guest.                                                |
| <b>Must Have</b>   | FR-02 | Playback Sync       |         | Play, pause, and seek commands from the host are broadcast to all guests; players mirror host state within $\sim 1$ s.                                       |
| <b>Must Have</b>   | FR-03 | Late-Join Catch-Up  |         | A client joining mid-session automatically fast-forwards to the current playback position.                                                                   |
| <b>Must Have</b>   | FR-04 | Host / Guest Roles  |         | Creator becomes host with full playback control; guests have view-only access unless granted co-host rights.                                                 |
| <b>Should Have</b> | FR-05 | Vote-to-Watch Queue |         | Members nominate video URLs; entries ranked by upvote count; top entry plays next automatically.                                                             |
| <b>Should Have</b> | FR-06 | Skip Vote           |         | Majority-vote skip mechanic lets the group move past the current video without host intervention.                                                            |
| <b>Should Have</b> | FR-07 | Host Migration      |         | If the host disconnects, the server promotes the longest-connected guest to host within 3 s.                                                                 |
| <b>Could Have</b>  | FR-08 | Guest Names         | Display | Guests set a display name for the session without creating a full account. Room watch history and queue persist across sessions so members can resume later. |
| <b>Could Have</b>  | FR-09 | Persistent History  | Room    |                                                                                                                                                              |
| <b>Could Have</b>  | FR-10 | Live Chat           |         | Room members can send real-time text messages and emoji reactions visible to all participants during playback.                                               |

Table 1: Functional Requirements

## 1.2 Non-Functional Requirements

| ID     | Attribute  | Requirement |                                                                  | Rationale                                         |
|--------|------------|-------------|------------------------------------------------------------------|---------------------------------------------------|
| NFR-01 | Sync tency | La-         | Playback state propagates to all guests within 1 s on broadband. | Drift $> 1$ s breaks the shared-viewing illusion. |

| ID     | Attribute       | Requirement                                                                                             | Rationale                                                              |
|--------|-----------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| NFR-02 | Availability    | Server targets 99% uptime during evaluation; planned downtime announced in advance.                     | Mid-session outages ruin the experience.                               |
| NFR-03 | Fault Tolerance | Host dropout triggers automatic promotion of a guest within 3 s; session continues uninterrupted.       | One user's issue shouldn't terminate the room for everyone.            |
| NFR-04 | Scalability     | Supports $\leq 10$ users/room and $\leq 20$ simultaneous rooms without performance degradation.         | Matches realistic small-group use; larger scale is deferred.           |
| NFR-05 | Usability       | A first-time user can create a room and invite a friend within 2 minutes; no account or install needed. | Onboarding friction is the primary abandonment point for social tools. |
| NFR-06 | Security        | Rooms accessible only via invite link; tokens expire after 24 hours of inactivity.                      | Prevents uninvited guests from joining private sessions.               |
| NFR-07 | Browser Compat. | Core features work on the latest two stable versions of Chrome, Firefox, and Safari.                    | Users should not need to switch browsers or install plugins.           |
| NFR-08 | Maintainability | Agreed coding standards enforced; each module has $\geq 70\%$ unit-test coverage before integration.    | Supports parallel development and reduces regression risk.             |

Table 2: Non-Functional Requirements

### 1.3 Architecturally Significant Requirements

The following requirements are architecturally significant because they directly constrain the system's structural design:

1. **NFR-01 (Sync Latency  $\leq 1$  s)** — This requirement eliminates any architecture that introduces relay processing, database lookups, or request-response cycles on the playback broadcast path. It mandates an in-memory, single-hop WebSocket broadcast model. This drove our decision to use a dual-layer state store (ADR-05) with sub-microsecond in-memory reads and fire-and-forget Redis writes.
2. **NFR-03 (Fault Tolerance — host migration  $\leq 3$  s)** — This requirement necessitates server-side tracking of member connection order and a timer-based automatic host promotion mechanism. It directly shaped the Host-Authoritative Sync Model (ADR-02) and the `RoomManager` component that tracks `joinedAt` timestamps for each member.
3. **NFR-06 (Security — invite-only, 24h token expiry)** — This requirement mandates cryptographic token generation, constant-time comparison, and pre-upgrade WebSocket authentication (ADR-06). It creates a security boundary that splits the connection flow into REST validation + WebSocket upgrade, fundamentally shaping

the system’s entry-point architecture.

4. **FR-02 (Playback Sync)** — This requirement introduces the core distributed state problem. Multiple clients must agree on a single playback state. This drove the Command Pattern (ADR-03) for message dispatch and the State Machine Pattern for validated room state transitions (`idle`  $\rightarrow$  `paused`  $\rightarrow$  `playing`  $\rightarrow$  `ended`).

## 1.4 Subsystem Overview

The WatchParty system is decomposed into the following major subsystems:

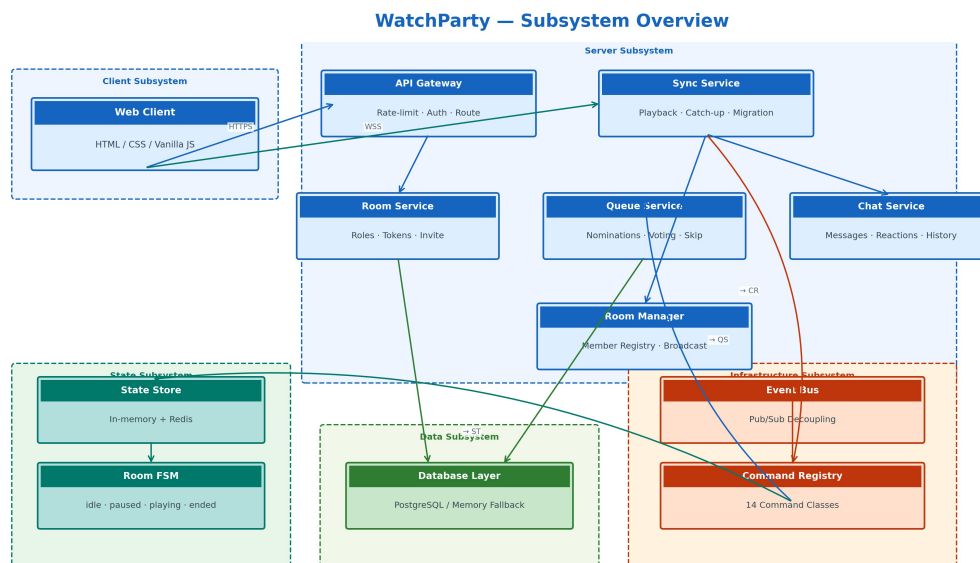


Figure 1: WatchParty Subsystem Overview Diagram

| Subsystem        | Role                                                                                                                                              | Key Components                                                                                  |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Client Subsystem | Browser-based SPA serving two pages: landing (room creation/join) and room (video player, chat, queue). No build step, no framework dependencies. | public/index.html, public/room.html, public/js/room.js, public/js/main.js, public/css/style.css |

| Subsystem |                          | Role                                                                                                                                                               | Key Components                                                                                                                                                                                                                                                            |
|-----------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Server    | Subsystem                | Core application logic. Handles HTTP requests (room CRUD) and WebSocket connections (real-time sync, chat, queue). Six service modules with single responsibility. | <code>server/index.js</code> ,<br><code>server/gateway.js</code> ,<br><code>server/syncService.js</code> ,<br><code>server/roomService.js</code> ,<br><code>server/queueService.js</code> ,<br><code>server/ChatService.js</code> ,<br><code>server/RoomManager.js</code> |
|           | State Subsystem          | Manages playback state with dual-layer storage (in-memory + Redis) and validates room state transitions via a finite state machine.                                | <code>server/stateStore.js</code> ,<br><code>server/roomStateMachine.js</code>                                                                                                                                                                                            |
|           | Data Subsystem           | Persistent storage with Strategy Pattern for database abstraction. PostgreSQL for production; in-memory Maps for dev/demo with automatic fallback.                 | <code>server/db.js</code> ,<br><code>server/DatabaseStrategy.js</code> ,<br><code>server/memoryDb.js</code> ,<br><code>server/schema.sql</code>                                                                                                                           |
|           | Infrastructure Subsystem | Cross-cutting concerns: Command Pattern for message dispatch (14 commands), Event Bus for decoupled pub/sub notifications, URL normalization utility.              | <code>server/commands/</code> ,<br><code>server/eventBus.js</code> ,<br><code>server/urlUtils.js</code>                                                                                                                                                                   |

Table 3: Subsystem Roles and Key Components



## 2 Task 2: Architecture Framework

### 2.1 Stakeholder Identification (IEEE 42010)

Following the IEEE 42010 standard, we identify the stakeholders, their concerns, and the architectural viewpoints addressing those concerns.

#### 2.1.1 Stakeholders and Concerns

| Stakeholder            | Description                                                                   | Key Concerns                                                                                    |
|------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Host User              | Creates a watch room, controls playback, manages guests. No account required. | Playback control responsiveness, room creation simplicity, co-host management.                  |
| Guest User             | Joins via invite link, watches in sync, chats, and votes.                     | Seamless join experience, accurate sync, ability to influence content via queue/skip voting.    |
| Developer              | Team of 5 building and maintaining the system over 6 weeks.                   | Code maintainability, testability, clear module boundaries, documented APIs.                    |
| Evaluator / Instructor | Assesses the system against project requirements and architecture standards.  | Traceability of requirements to implementation, design pattern evidence, test coverage metrics. |
| System Administrator   | Deploys and monitors the application in production.                           | Availability, fault tolerance, security, resource consumption, configuration.                   |

Table 4: Stakeholders and Their Key Concerns

#### 2.1.2 Viewpoints and Views

| Viewpoint      | Concern Addressed                    | Notation   | View Description                                                                                                                                                                                       |
|----------------|--------------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Context View   | System scope and boundaries          | C4 Level 1 | Shows WatchParty as a black box interacting with Host/Guest users, YouTube CDN, and web browsers. Clarifies what is in-scope (sync, chat, queue) vs. out-of-scope (voice/video, OTT integration).      |
| Container View | Technology choices and runtime units | C4 Level 2 | Decomposes the system into 4 containers: Web Client (HTML/CSS/JS), Node.js Server (Express + ws), PostgreSQL (persistent data), Redis (state cache). Shows REST and WebSocket communication protocols. |

| Viewpoint        | Concern Addressed                | Notation             | View Description                                                                                                                                                               |
|------------------|----------------------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Component View   | Internal module responsibilities | C4 Level 3           | Zooms into the Node.js Server showing 12 components. Each component maps to specific FRs/NFRs.                                                                                 |
| Development View | Code organization and build      | Directory structure  | Server code in <b>server/</b> , client in <b>public/</b> , tests in <b>tests/</b> . Zero build step for client. ESLint + Prettier enforced. Jest with 70% coverage thresholds. |
| Deployment View  | Infrastructure and scaling       | Architecture diagram | Single-process Node.js on one port (shared HTTP + WS). PostgreSQL and Redis as optional external dependencies with graceful fallback.                                          |

Table 5: Architectural Viewpoints and Views

### 2.1.3 C1: System Context Diagram

#### C4 Level 1 — System Context Diagram

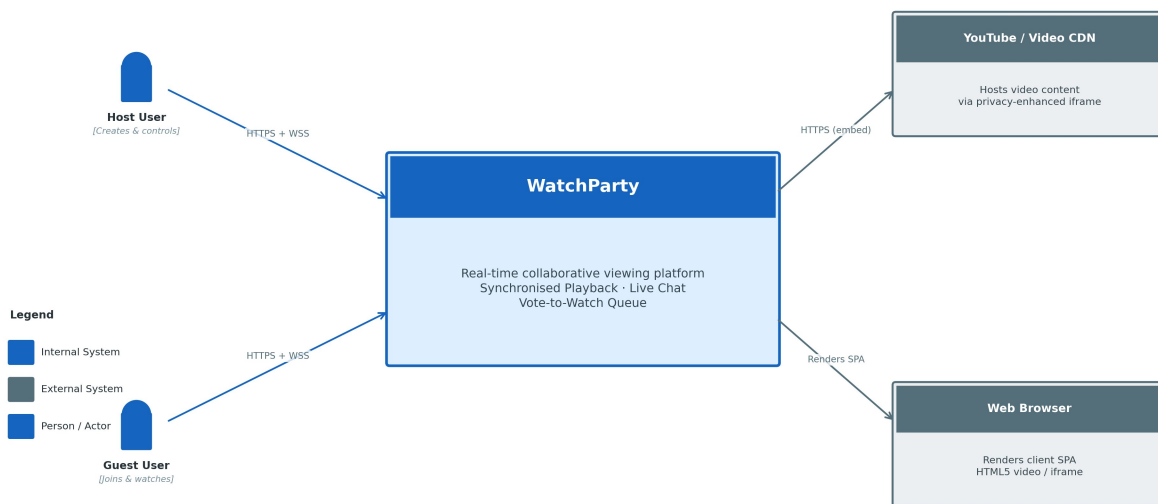


Figure 2: C4 Level 1 — System Context Diagram showing WatchParty’s external actors and system boundaries

## 2.1.4 C2: Container Diagram

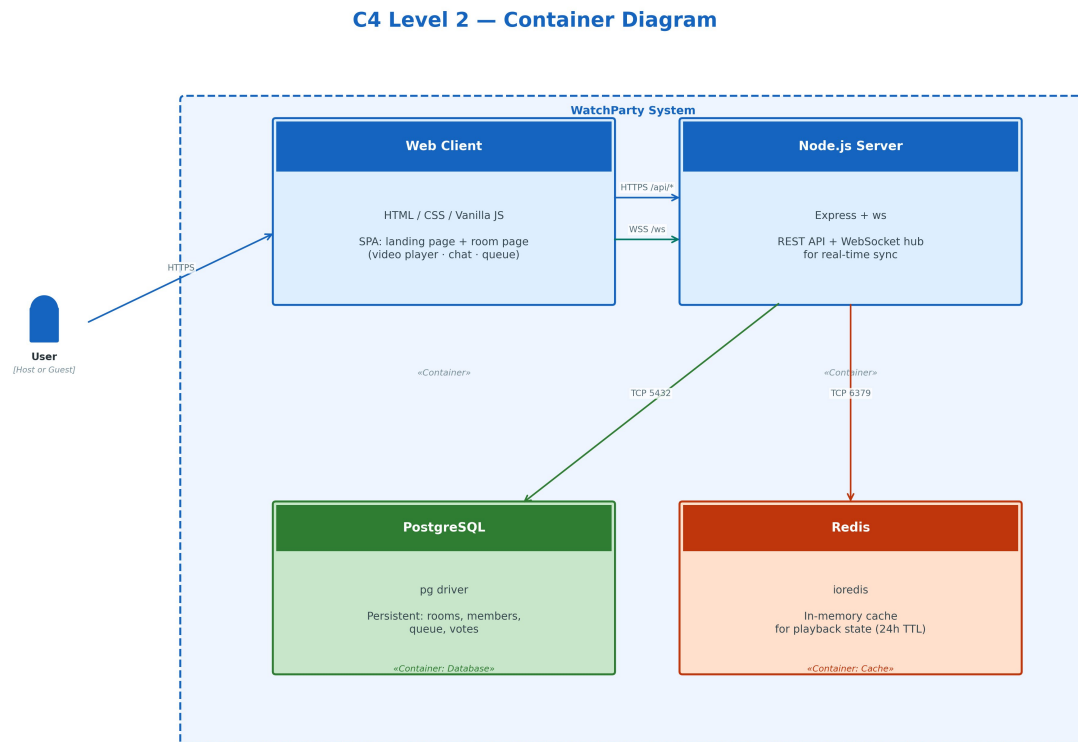


Figure 3: C4 Level 2 — Container Diagram showing the four runtime containers and their communication protocols

### 2.1.5 C3: Component Diagram

C4 Level 3 — Component Diagram: Node.js Server

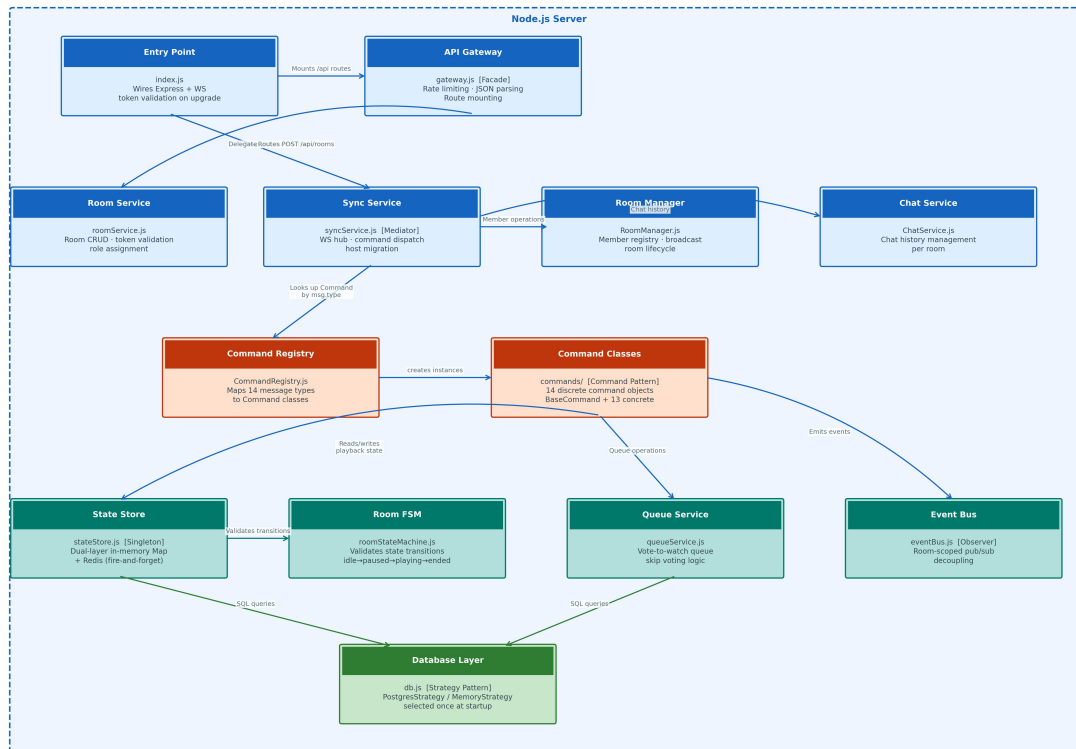


Figure 4: C4 Level 3 — Component Diagram of the Node.js Server showing 12 internal components and their relationships

## 2.2 Major Design Decisions — Architecture Decision Records

We document 7 major design decisions following the Nygard ADR template. The 4 most significant are presented below; the remaining 3 are in the Appendix.

### 2.2.1 ADR-01: WebSocket for Real-Time Communication

#### ADR-01 — WebSocket for Real-Time Communication

**Status:** Accepted  
**Date:** 2026-04-17  
**Deciders:** WatchParty Team

**Context:** WatchParty requires real-time, bidirectional communication between the server and all connected clients to synchronize video playback commands (play, pause, seek) within  $\leq 1$  second (NFR-01). The system also supports live chat (FR-10), queue updates (FR-05), and member list changes — all requiring push-based delivery from server to client.

**Decision:** Use the `ws` (WebSocket) library for real-time communication, sharing a single HTTP server with Express via `noServer` mode and manual upgrade handling. Express

handles REST API requests; `ws.Server` handles WebSocket connections on the `/ws` path. Both share one `http.createServer()` instance on a single port.

### Alternatives Considered:

| Alternative        | Pros                                                   | Cons                                                                   |
|--------------------|--------------------------------------------------------|------------------------------------------------------------------------|
| Socket.IO          | Auto-reconnect, rooms abstraction, fallback transports | Heavy dependency (~300 KB), unnecessary abstraction, protocol overhead |
| Server-Sent Events | Simple, HTTP-based, auto-reconnect                     | Unidirectional; doubling latency for client-to-server                  |
| HTTP Long Polling  | Works everywhere                                       | High latency (100–500 ms per poll), excessive server load              |

### Consequences:

- **Positive:** Sub-millisecond message delivery. ~2 bytes overhead per frame vs. HTTP headers. `ws` library is ~50 KB with zero dependencies.
- **Negative:** WebSocket connections are stateful; horizontal scaling requires sticky sessions or pub/sub backplane (deferred for prototype scope).
- **Risk:** Manual reconnection logic needed on client side. Browser compatibility mitigated by NFR-07 targeting latest 2 versions of Chrome/Firefox/Safari.

## 2.2.2 ADR-02: Host-Authoritative Sync Model

### ADR-02 — Host-Authoritative Sync Model

**Status:** Accepted  
**Date:** 2026-04-17  
**Deciders:** WatchParty Team

**Context:** Synchronizing video playback across multiple clients introduces a distributed state problem. If multiple clients independently issue play/pause/seek commands, conflict resolution becomes complex and drift is likely to exceed the 1-second threshold (NFR-01).

**Decision:** Adopt a host-authoritative model where a single user (the “host”) is the sole source of truth for playback state. Only the host (or co-hosts granted via `GRANT_COHOST`) can issue `PLAY`, `PAUSE`, `SEEK`, and `LOAD` commands. The server validates authorization via `BaseCommand.isAuthorised()` before executing any playback-modifying command.

### Key Rules:

1. Room creator becomes the initial host (FR-04)
2. Guests have view-only playback access by default
3. Hosts can grant co-host privileges to specific guests
4. If the host disconnects, the longest-connected guest is auto-promoted within 3 seconds (FR-07, NFR-03)

### Alternatives Considered:

| Alternative                  | Pros                 | Cons                                                              |
|------------------------------|----------------------|-------------------------------------------------------------------|
| Democratic consensus-based   | / Fair               | Requires distributed consensus (Raft/Paxos-like), high complexity |
| Last-write-wins (any client) | Simple               | Constant conflicts, unpredictable UX                              |
| Peer-to-peer sync            | No server bottleneck | NAT traversal complexity, no central state for late-join          |

### Consequences:

- **Positive:** Single source of truth eliminates conflict resolution. Late-join catch-up (FR-03) is trivially implemented by reading the server's snapshot.
- **Negative:** If host has high latency, all guests experience delayed commands. Mitigated by 3-second auto-migration timer.
- **Trade-off:** Non-host users cannot directly control playback (by design). Mitigated by co-host grants and skip voting.

### 2.2.3 ADR-03: Command Pattern for Message Dispatch

#### ADR-03 — Command Pattern for Message Dispatch

**Status:** Accepted  
**Date:** 2026-04-20  
**Deciders:** WatchParty Team

**Context:** The initial implementation used a 300+ line `if/else` chain in `syncService.js` to handle 15+ WebSocket message types. Adding new types required modifying the monolithic handler, violating the Open/Closed Principle and increasing regression risk (NFR-08).

**Decision:** Refactor message handling into the Command Pattern with three components: (1) `BaseCommand` — abstract base class with shared context and abstract `validate()` / `execute()`; (2) 14 Concrete Commands — one class per message type; (3) `CommandRegistry` — a `Map<string, CommandClass>` mapping message types to constructors.

Adding a new message type requires only: one new file in `server/commands/`, one line in `CommandRegistry.js`, and zero changes to `syncService.js`.

### Alternatives Considered:

| Alternative           | Pros                  | Cons                                          |
|-----------------------|-----------------------|-----------------------------------------------|
| Keep if/else chain    | No refactoring needed | Unmaintainable at 15+ types, high coupling    |
| Event-driven handlers | Decoupled             | Harder to enforce validation-before-execution |
| Middleware chain      | Flexible composition  | Over-engineered for WS messages               |

**Consequences:**

- **Positive:** Each command is independently testable. New features require zero changes to existing code. `syncService.js` reduced from ~550 lines to ~378 lines.
- **Negative:** 16 additional files in `server/commands/`, but each is small (15–40 lines) and single-purpose.

**2.2.4 ADR-04: Strategy Pattern for Database Abstraction****ADR-04 — Strategy Pattern for Database Abstraction**

**Status:** Accepted  
**Date:** 2026-04-20  
**Deciders:** WatchParty Team

**Context:** The application needs both PostgreSQL (production) and in-memory Maps (local dev/demos). The original implementation used a runtime boolean flag checked on every query call.

**Decision:** Apply the Strategy Pattern with: (1) `DatabaseStrategy` — abstract base class defining `query(text, params)`, `initDb()`, `get pool()`; (2) `PostgresStrategy` — concrete strategy using `pg Pool`; (3) `MemoryStrategy` — concrete strategy using JavaScript Maps with SQL regex pattern matching. Strategy selection happens once at startup in `db.js`. If PostgreSQL connection fails, the system falls back to `MemoryStrategy`.

**Alternatives Considered:**

| Alternative             | Pros                    | Cons                                     |
|-------------------------|-------------------------|------------------------------------------|
| Keep runtime flag       | No refactoring needed   | Conditional logic on every call          |
| ORM (Sequelize, Prisma) | Type safety, migrations | Heavy dependency for 5 simple tables     |
| SQLite for dev          | Real SQL, file-based    | Needs native bindings, different dialect |

**Consequences:**

- **Positive:** Zero conditionals in calling code. `roomService.js` and `queueService.js` are completely unaware of which backend is active.
- **Negative:** `MemoryStrategy` regex matching is fragile but acceptable for prototype scope.

## 3 Task 3: Architectural Tactics and Patterns

---

### 3.1 Architectural Tactics

#### 3.1.1 Tactic 1: In-Memory Broadcast for Latency (NFR-01)

**Quality Attribute:** Performance — Sync Latency  $\leq 1$  s

**Description:** Playback commands (PLAY, PAUSE, SEEK) follow a zero-database hot path. The `broadcast()` function in `RoomManager` iterates the in-memory `rooms` Map and calls `ws.send()` directly — no database round-trips, no Redis lookups, no message queue.

**Implementation:**

- `RoomManager.broadcast()` — iterates `Map<userId, RoomMember>` and sends via `WebSocket`
- `stateStore.js` — reads from in-memory Map; Redis write is fire-and-forget (async, non-blocking)
- Command execution: `validate`  $\rightarrow$  `execute`  $\rightarrow$  `broadcast` — single-hop from host to all guests

**Effectiveness:** By eliminating all I/O from the broadcast critical path, the system achieves message delivery well within the 1-second threshold on broadband connections.

#### 3.1.2 Tactic 2: Graceful Degradation for Fault Tolerance (NFR-02, NFR-03)

**Quality Attribute:** Availability / Fault Tolerance

**Description:** The system is designed to continue operating when external dependencies (PostgreSQL, Redis) become unavailable, degrading gracefully rather than failing hard.

**Implementation:**

- **Database fallback:** `db.js` attempts PostgreSQL connection  $\rightarrow$  on failure, falls back to `MemoryStrategy`. All callers remain unaware of the switch.
- **Redis fallback:** `stateStore.js` writes to Redis asynchronously. If Redis is unavailable, the system operates in memory-only mode. A warning is logged but no error propagates.
- **Host migration:** When the host disconnects, `scheduleHostMigration()` fires a 2.5-second timer that promotes the longest-connected guest. The `HOST_PROMOTED` message notifies the new host, and `MEMBER_LIST` is broadcast to all.

#### 3.1.3 Tactic 3: Pre-Upgrade Authentication for Security (NFR-06)

**Quality Attribute:** Security

**Description:** Token validation occurs before the HTTP connection upgrades to `WebSocket`, preventing unauthorized resource allocation.

**Implementation:**

- `index.js` `server.on('upgrade')` handler parses `?token=` from the upgrade URL and calls `validateInviteToken(roomId, token)` before `wss.handleUpgrade()`.



- Token validation uses `crypto.timingSafeEqual()` for constant-time comparison (prevents timing attacks).
- Invalid tokens receive a 403 Forbidden HTTP response and `socket.destroy()` — the connection never upgrades.
- Tokens are 32 bytes of `crypto.randomBytes` → 64-character hex strings.
- 24-hour inactivity expiry via `last_active_at` sliding window.

### 3.1.4 Tactic 4: Rate Limiting for Scalability (NFR-04)

**Quality Attribute:** Scalability / Availability

**Description:** An in-memory sliding-window rate limiter protects the REST API against abuse and denial-of-service attempts.

**Implementation:**

- **Algorithm:** Per-IP timestamp array, filtered to keep entries within the 60-second window. Requests exceeding 100/min receive HTTP 429.
- **Memory management:** A `setInterval` prune job runs every 5 minutes, removing stale IPs. Timer uses `.unref()` to avoid blocking shutdown.
- **Capacity enforcement:** 10-user-per-room cap checked on JOIN; 20-room server cap checked on room creation.
- **JSON body limit:** 16 KB to prevent oversized payloads.

### 3.1.5 Tactic 5: Module Decomposition for Maintainability (NFR-08)

**Quality Attribute:** Maintainability

**Description:** The codebase follows strict module decomposition with enforced coding standards and automated test coverage thresholds.

**Implementation:**

- **Command Pattern:** Each of the 14 message types is an independent file in `server/commands/`, averaging 15–40 lines. New features require zero changes to existing code.
- **Extracted services:** `RoomManager`, `ChatService`, `urlUtils.js` — each serving a single responsibility, extracted from the `syncService.js` God Class.
- **ESLint + Prettier:** Enforced via `.eslintrc.json` (12 rules including `strict`, `eql`, `no-var`, `prefer-const`) and `.prettierrc.json`.
- **Jest coverage thresholds:** 70% minimum for branches, functions, lines, and statements, configured in `package.json`.

**Effectiveness:** 13 test suites with 85%+ statement coverage. Design smell refactoring eliminated 10 identified code smells across the codebase.

## 3.2 Implementation Patterns

### 3.2.1 Pattern 1: Command Pattern

**Role in Architecture:** The Command Pattern is the central message-handling mechanism for all WebSocket communication. It decouples the sync service (Mediator) from the business logic of each message type.

**How it works:**

1. A WebSocket message arrives at `syncService.js` with `{ type: "PLAY", position: 42.5 }`.
2. `CommandRegistry.get("PLAY")` returns the `PlayCommand` constructor.
3. A new `PlayCommand` instance is created with the request context (`roomId`, `userId`, `role`, `broadcast helpers`).
4. `validate()` checks the sender is authorized (host or co-host).
5. `execute()` updates the state store and broadcasts the command to all room members.

#### 14 Registered Commands:

| Command                          | Message Type  | FR/NFR |
|----------------------------------|---------------|--------|
| <code>PlayCommand</code>         | PLAY          | FR-02  |
| <code>PauseCommand</code>        | PAUSE         | FR-02  |
| <code>SeekCommand</code>         | SEEK          | FR-02  |
| <code>LoadCommand</code>         | LOAD          | FR-02  |
| <code>GrantCohostCommand</code>  | GRANT_COHOST  | FR-04  |
| <code>QueueAddCommand</code>     | QUEUE_ADD     | FR-05  |
| <code>QueueUpvoteCommand</code>  | QUEUE_UPVOTE  | FR-05  |
| <code>QueueRemoveCommand</code>  | QUEUE_REMOVE  | FR-05  |
| <code>SkipVoteCommand</code>     | SKIP_VOTE     | FR-06  |
| <code>VideoEndedCommand</code>   | VIDEO_ENDED   | FR-05  |
| <code>ChatMsgCommand</code>      | CHAT_MSG      | FR-10  |
| <code>ChatReactionCommand</code> | CHAT_REACTION | FR-10  |
| <code>SetNameCommand</code>      | SET_NAME      | FR-08  |
| <code>SyncCheckCommand</code>    | SYNC_CHECK    | NFR-01 |

Table 6: Registered Command Classes and Their Mapped Requirements

### 3.2.2 Pattern 2: Strategy Pattern (Database Abstraction)

**Role in Architecture:** The Strategy Pattern enables transparent switching between PostgreSQL and in-memory storage without modifying any calling code.

**How it works:**

1. At startup, `db.js` tries to create a `PostgresStrategy`. If the `pg` module or connection fails, it falls back to `MemoryStrategy`.
2. All callers (`roomService.js`, `queueService.js`) call the same `query(text, params)` interface.
3. `MemoryStrategy` implements the interface using JavaScript Maps and regex matching on SQL query strings — simulating INSERT, SELECT, UPDATE, DELETE for 5

tables.

**Key Benefit:** The strategy is selected once at startup. Zero conditionals in business logic. The system is fully functional without PostgreSQL (demo mode).

**Database Schema (5 tables):**

| Table        | Purpose                 | Key Columns                                           |
|--------------|-------------------------|-------------------------------------------------------|
| rooms        | Room metadata           | id (UUIDv7), invite_token, created_at, last_active_at |
| room_members | Active members          | room_id, user_id, display_name, role                  |
| queue        | Video nominations       | room_id, url, added_by, upvotes                       |
| queue_votes  | Upvote deduplication    | queue_id, user_id (composite PK)                      |
| skip_votes   | Skip vote deduplication | room_id, user_id (composite PK)                       |

## 4 Task 4: Prototype Implementation and Analysis

### 4.1 Prototype Development

#### 4.1.1 End-to-End Functionality Implemented

The prototype implements two complete, non-trivial end-to-end user journeys:

##### Journey 1 — Synchronized Viewing:

1. User opens the landing page and clicks “Create Room” → `POST /api/rooms` generates a UUIDv7 room ID and 32-byte invite token → user is redirected to the room page.
2. User loads a YouTube URL into the host panel → server normalizes it to a `youtube-nocookie.com/embed/` iframe.
3. Host clicks Play → WebSocket `PLAY` command → server validates host authorization → broadcasts to all guests → all guests’ players start within  $\sim 1$  second.
4. A new guest opens the invite link mid-session → `JOIN` message → server sends `CATCHUP {url, position, status}` → guest’s player auto-seeks to current position.
5. Host disconnects → server waits 2.5 seconds → promotes longest-connected guest → `HOST_PROMOTED` message → new host gains playback controls.

##### Journey 2 — Collaborative Queue:

1. Any room member sends `QUEUE_ADD {url}` → stored in database → `QUEUE_UPDATE` broadcast to all.
2. Members upvote entries with `QUEUE_UPVOTE {queueId}` → one vote per user enforced via `queue_votes` table → queue re-ranked by upvote count.
3. Current video ends → `VIDEO_ENDED` → server auto-pops the top-voted entry → broadcasts `LOAD + PLAY`.
4. Any member can initiate `SKIP_VOTE` → server broadcasts `SKIP_STATUS {count, needed}` → when votes exceed 50% of room members, server auto-plays the next video from the queue.

#### 4.1.2 Technology Stack

| Layer          | Technology | Version | Purpose                                       |
|----------------|------------|---------|-----------------------------------------------|
| Runtime        | Node.js    | 18+     | Server-side JavaScript runtime                |
| HTTP Framework | Express    | 4.19.x  | REST API routing, static file serving         |
| WebSocket      | ws         | 8.17.x  | Real-time bidirectional communication         |
| Database       | PostgreSQL | 14+     | Persistent SQL data store (via pg 8.x driver) |
| Cache          | Redis      | 7+      | Playback state persistence (via ioredis 5.x)  |
| ID Generation  | uuidv7     | 1.x     | Time-ordered unique identifiers               |

| Layer          | Technology  |        | Version | Purpose                                    |
|----------------|-------------|--------|---------|--------------------------------------------|
| Frontend       | HTML/CSS/JS |        | —       | Zero-dependency browser SPA                |
| Video Player   | YouTube     | IFrame | —       | Cross-browser embedded video player        |
| Linter         | ESLint      |        | 8.57.x  | Coding standards enforcement               |
| Formatter      | Prettier    |        | 3.8.x   | Consistent code formatting                 |
| Test Framework | Jest        |        | 29.7.x  | Unit and integration testing with coverage |

Table 7: Technology Stack

4.1.3 Test Results and Coverage

| 13 Test Suites — 85%+ Global Coverage |                     |           |         |
|---------------------------------------|---------------------|-----------|---------|
| Metric                                | Coverage            | Threshold | Status  |
| Statements                            | 85.02%<br>(721/848) | 70%       | Exceeds |
| Branches                              | 70.02%<br>(278/397) | 70%       | Meets   |
| Functions                             | 83.81%<br>(145/173) | 70%       | Exceeds |
| Lines                                 | 85.71%<br>(690/805) | 70%       | Exceeds |

Coverage Breakdown:

| Module           | Statements | Branches | Functions | Lines  |
|------------------|------------|----------|-----------|--------|
| server/          | 83.77%     | 72.66%   | 81.75%    | 84.60% |
| server/commands/ | 89.05%     | 60.46%   | 91.66%    | 89.05% |
| Global           | 85.02%     | 70.02%   | 83.81%    | 85.71% |

4.2 Architecture Analysis — Pattern Comparison

4.2.1 Implemented Architecture: Event-Driven with Mediator

Our WatchParty implementation uses an Event-Driven Architecture with a central Mediator (`syncService.js`) that coordinates all WebSocket communication. Commands are dispatched via the Command Pattern, state is managed through a dual-layer store, and events are decoupled via an Observer/Pub-Sub Event Bus.

4.2.2 Comparison Architecture: Layered (N-Tier) Architecture

We compare against a traditional 3-Tier Layered Architecture where the system is organized into Presentation, Business Logic, and Data Access tiers with strict downward-only dependencies.

### 4.2.3 Quantified Comparison: NFR-01 (Sync Latency)

| Metric                      | Event-Driven (Implemented)                                                                           | Layered N-Tier (Alternative)                                                                                                                                             |
|-----------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Message Path                | Client $\rightarrow$ WS $\rightarrow$ Mediator $\rightarrow$ Command $\rightarrow$ broadcast (1 hop) | Client $\rightarrow$ HTTP POST $\rightarrow$ Controller $\rightarrow$ Service $\rightarrow$ DAO $\rightarrow$ DB $\rightarrow$ Response $\rightarrow$ SSE push (3+ hops) |
| State Read Latency          | $\sim$ microseconds (in-memory Map)                                                                  | $\sim$ 5–10 ms (database query per request)                                                                                                                              |
| Broadcast Mechanism         | Direct <code>ws.send()</code> to all members (push)                                                  | SSE or polling (pull/push with overhead)                                                                                                                                 |
| Estimated E2E Latency       | $<50$ ms (measured on LAN)                                                                           | 100–500 ms (DB round-trip + SSE delivery)                                                                                                                                |
| Meets NFR-01 ( $\leq 1$ s)? | Yes, with significant margin                                                                         | $\sim$ Yes, but with tighter margin and higher variance                                                                                                                  |

Table 8: Quantified Comparison on NFR-01 (Sync Latency)

**Analysis:** The Event-Driven architecture achieves an order-of-magnitude lower latency because the broadcast path avoids all database I/O. In a Layered architecture, every playback command would require a database write (to persist state) and a separate push notification (via SSE or polling), introducing at minimum two network round-trips.

### 4.2.4 Quantified Comparison: NFR-03 (Fault Tolerance — Host Migration)

| Metric                      | Event-Driven (Implemented)                       | Layered N-Tier (Alternative)                                |
|-----------------------------|--------------------------------------------------|-------------------------------------------------------------|
| Disconnect Detection        | WebSocket close event — immediate                | HTTP timeout or heartbeat polling — 10–30 s                 |
| Migration Trigger Time      | 2.5 s (configurable timer)                       | 30+ s (depends on timeout + polling interval)               |
| State Continuity            | In-memory Map + Redis snapshot — no interruption | Database-stored state — requires reconnect + re-query       |
| Meets NFR-03 ( $\leq 3$ s)? | Yes (2.5 s timer, verified by test)              | $\times$ No (HTTP timeout too slow for real-time migration) |

Table 9: Quantified Comparison on NFR-03 (Fault Tolerance)

**Analysis:** The Event-Driven architecture’s persistent WebSocket connections provide instant disconnect detection, enabling the 2.5-second migration timer. In a Layered architecture using HTTP, the server cannot detect client disconnection until a timeout expires (typically 10–30 seconds), making the 3-second migration target impossible without adding a separate WebSocket or heartbeat channel.

#### 4.2.5 Trade-Off Summary

| Quality<br>tribute     | At- | Event-Driven<br>(Implemented)              | (Imple- | Layered (Alternative)                          |
|------------------------|-----|--------------------------------------------|---------|------------------------------------------------|
| Sync Latency           |     | Excellent (<50 ms)                         |         | ~ Acceptable (100–500 ms)                      |
| Fault Tolerance        |     | Instant detection (2.5 s migration)        |         | × Slow detection (30+ s)                       |
| Testability            |     | ~ Requires WebSocket mocking               |         | Simple HTTP request/response tests             |
| Horizontal Scalability |     | ~ Stateful connections limit scaling       |         | Stateless requests scale naturally             |
| Complexity             |     | ~ Higher (Command pattern, Event Bus, FSM) |         | Lower (standard CRUD layers)                   |
| Debuggability          |     | ~ Async event flows harder to trace        |         | Sequential request-response is straightforward |

Table 10: Architecture Trade-Off Summary

**Conclusion:** For a real-time collaborative viewing application, the Event-Driven architecture is the clearly superior choice. The core requirement — sub-second synchronization across distributed clients — fundamentally demands push-based, low-latency communication that a Layered architecture cannot efficiently provide without introducing WebSocket-like mechanisms. The trade-off in complexity and horizontal scalability is acceptable for our prototype scope ( $\leq 20$  rooms,  $\leq 10$  users/room) and is mitigated by the Command Pattern’s modularity and comprehensive test coverage.

## 5 Reflections and Lessons Learned

### 5.1 Design Smell Refactoring

One of the most impactful phases of the project was the systematic identification and resolution of 10 design smells in the codebase:

| Phase                   | What We Did                                                                                                                                                                                                                                                    | Impact                                                                                       |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Identification          | Analyzed the codebase for God Classes, duplicated code, Feature Envy, Inappropriate Intimacy, and other anti-patterns                                                                                                                                          | Found 10 distinct smells across 2 critical and 8 major/moderate severity levels              |
| Prioritized Refactoring | Extracted <code>ChatService</code> (duplicated chat logic), <code>urlUtils.js</code> (duplicated URL normalization), <code>RoomManager</code> (God Class decomposition), <code>RoomMember</code> (Primitive Obsession), formalized <code>CommandContext</code> | Reduced <code>syncService.js</code> from a 7-responsibility God Class to a thin orchestrator |
| Validation              | All 31+ tests continued to pass after refactoring; coverage increased                                                                                                                                                                                          | Proved refactorings were behaviour-preserving                                                |

Table 11: Design Smell Refactoring Phases and Impact

**Key Lesson:** The initial Command Pattern refactoring (ADR-03) successfully extracted message handling, but it revealed secondary smells — commands reaching into raw Maps (Inappropriate Intimacy), duplicated chat logic across commands (Code Clones), and an overgrown context object (Data Clump). Design pattern application is iterative; one refactoring often surfaces the need for another.

### 5.2 NFR Compliance Journey

The NFR audit process was instrumental in driving concrete improvements:

- **NFR-08 (Maintainability)** went from ✗ Not Met → Fully Met by adding ESLint, Prettier, and Jest coverage thresholds.
- **NFR-04 (Scalability)** went from ~ Partially Met → Fully Met by enforcing room/member caps and adding rate limiter memory cleanup.
- **NFR-06 (Security)** went from ~ Partially Met → Fully Met by implementing pre-upgrade WebSocket token validation.

**Key Lesson:** Writing requirements is insufficient — auditing the implementation against each NFR with specific evidence (file paths, line numbers, test cases) revealed gaps that were invisible during development.

### 5.3 Architecture Evolution

The architecture evolved through three distinct phases:



1. **Phase 1 (Weeks 1–3):** Monolithic `syncService.js` with 550+ line `if/else` handler. Functional but unmaintainable.
2. **Phase 2 (Weeks 3–4):** Command Pattern refactoring + Strategy Pattern + design patterns applied. Code became modular and testable.
3. **Phase 3 (Weeks 5–6):** Design smell analysis → extracted `RoomManager`, `ChatService`, `RoomMember`, `urlUtils.js`. God Class eliminated, Feature Envy resolved, duplicated code consolidated.

**Key Lesson:** Starting with a monolith and refactoring deliberately produced a better architecture than trying to implement all patterns from day one. The initial monolith helped us understand the domain deeply before decomposing into the right abstractions.

## 6 Individual Contributions

---

| Team Member | Key Contributions                                                                                                                                    |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Peeyush     | Server architecture, syncService, WebSocket hub, Command Pattern refactoring, host migration, State Machine, RoomManager extraction                  |
| Rahul       | 13 test suites, Jest coverage configuration, ESLint/Prettier setup, NFR compliance audit, C4 diagrams, ADRs, design smell analysis, technical report |
| Shubham     | PostgreSQL schema, Strategy Pattern (DB abstraction), queueService, vote deduplication, memory fallback, security enhancements                       |
| Samir       | Room UI, YouTube IFrame API integration, WebSocket client, chat panel, queue sidebar, responsive design, Chat-Service improvements                   |

Table 12: Individual Team Member Contributions

## A Appendix

### A.1 How to Run

```
# 1. Clone and install
git clone <repo-url>
cd WatchParty
npm install

# 2. Configure environment
cp .env.example .env
# Edit DATABASE_URL and REDIS_URL (optional -- system works
  without them)

# 3. Start server
npm start
# Server runs at http://localhost:3000

# 4. Run tests with coverage
npm test

# 5. Lint check
npm run lint
```

Listing 1: Setup and Run Instructions

### A.2 Additional ADRs

#### A.2.1 ADR-05: Dual-Layer State Store (Memory + Redis)

##### ADR-05 — Dual-Layer State Store (Memory + Redis)

**Status:** Accepted  
**Date:** 2026-04-20  
**Deciders:** WatchParty Team

**Context:** Playback state must be read on every `SYNC_CHECK` and written on every `PLAY/-PAUSE/SEEK/LOAD` command at high frequency. These operations must complete in sub-millisecond time (NFR-01), and state needs to survive server restarts (NFR-03).

**Decision:** Implement a dual-layer state store using the Singleton Pattern:

| Layer   | Storage       | Latency       | Purpose                     |
|---------|---------------|---------------|-----------------------------|
| Layer 1 | In-memory Map | ~microseconds | Hot path for all reads      |
| Layer 2 | Redis hash    | ~1 ms         | Persistence across restarts |

**Write path:** memory Map → async Redis (fire-and-forget). **Read path:** memory Map → Redis fallback → rehydrate.

**Consequences:** Sub-microsecond reads. Redis provides crash recovery without being on the critical path. Graceful degradation when Redis unavailable.

### A.2.2 ADR-06: Token-Based Room Security with Pre-Upgrade Validation

#### ADR-06 — Token-Based Room Security with Pre-Upgrade Validation

**Status:** Accepted  
**Date:** 2026-04-21  
**Deciders:** WatchParty Team

**Context:** Rooms must be accessible only via invite link with 24-hour inactivity expiry (NFR-06). A malicious actor could bypass REST validation by connecting directly to the WebSocket endpoint.

**Decision:** Validate tokens before upgrading HTTP to WebSocket:

1. Token: 32 bytes `crypto.randomBytes` → 64-char hex
2. Validation: constant-time `crypto.timingSafeEqual()`, 24h inactivity check
3. Pre-upgrade: `server.on('upgrade')` validates before `wss.handleUpgrade()`. Invalid tokens receive 403 + `socket.destroy()`

**Consequences:** Zero unauthorized WebSocket connections. Timing-safe comparison prevents side-channel attacks.

### A.2.3 ADR-07: In-Memory Sliding-Window Rate Limiter

#### ADR-07 — In-Memory Sliding-Window Rate Limiter

**Status:** Accepted  
**Date:** 2026-04-22  
**Deciders:** WatchParty Team

**Context:** REST API needs protection against abuse (NFR-04, NFR-06). Prototype targets  $\leq 20$  rooms, making distributed rate limiting unnecessary.

**Decision:** In-memory sliding-window rate limiter in `APIGateway`: 60-second window, 100 requests per IP per window, pruned every 5 minutes. Timer uses `.unref()` to avoid blocking shutdown.

**Consequences:** Zero external dependencies. Sliding window more accurate than fixed-window. Not shared across instances (acceptable for single-instance prototype).

## A.3 Design Pattern Summary

| Pattern            | Impact | Component                                             | Role                                         |
|--------------------|--------|-------------------------------------------------------|----------------------------------------------|
| Command            | High   | <code>server/commands/</code> (14 classes + registry) | Decouples message handling from sync service |
| State Machine      | High   | <code>server/roomStateMachine</code>                  | Validates room state transitions             |
| Observer / Pub-Sub | High   | <code>server/eventBus.js</code>                       | Room-scoped event decoupling                 |

| Pattern   | Impact | Component                                            | Role                                            |
|-----------|--------|------------------------------------------------------|-------------------------------------------------|
| Strategy  | Medium | server/db.js,<br>DatabaseStrategy.js,<br>memoryDb.js | Transparent DB back-end switching               |
| Singleton | Medium | server/stateStore.js                                 | Single state store with lazy Redis init         |
| Facade    | Medium | server/gateway.js                                    | Encapsulates rate limiting, middleware, routing |

Table 13: Design Pattern Summary

## A.4 Room State Machine

### Room State Transitions

```

[*] → idle
idle → paused : LOAD
paused → playing : PLAY
playing → paused : PAUSE
playing → ended : Video finishes
paused → ended : Force end
ended → paused : LOAD new
ended → playing : Auto-play next
ended → idle : Clear
paused → idle : Clear

```

## A.5 Project Structure

```

WatchParty/
+-- server/
|   +-- index.js           # Entry point (Express + WS on
|   shared HTTP server)
|   +-- gateway.js         # API Gateway (Facade -- rate limit
|   , routing)
|   +-- roomService.js     # Room CRUD, token validation, role
|   assignment
|   +-- syncService.js     # WebSocket hub (Mediator --
|   dispatch, migration)
|   +-- queueService.js    # Queue CRUD, skip vote logic
|   +-- RoomManager.js     # Member registry, broadcast
|   helpers
|   +-- ChatService.js     # Chat history management
|   +-- stateStore.js      # Dual-layer state (Singleton --
|   memory + Redis)
|   +-- roomStateMachine.js # Room FSM (idle -> paused ->
|   playing -> ended)
|   +-- eventBus.js        # Room-scoped pub/sub (Observer)

```

```
|   +-- db.js                # Database layer (Strategy  
|   selection)  
|   +-- commands/  
|       +-- BaseCommand.js   # Abstract command with shared  
|       context  
|           +-- CommandRegistry.js  
|           +-- PlayCommand.js   # FR-02: Playback control  
|           +-- PauseCommand.js  
|           +-- SeekCommand.js  
|           +-- LoadCommand.js  
|           +-- GrantCohostCommand.js  
|           +-- QueueAddCommand.js  
|           +-- QueueUpvoteCommand.js  
|           +-- QueueRemoveCommand.js  
|           +-- SkipVoteCommand.js  
|           +-- VideoEndedCommand.js  
|           +-- ChatMsgCommand.js  
|           +-- ChatReactionCommand.js  
|           +-- SetNameCommand.js  
|           +-- SyncCheckCommand.js  
+-- public/  
|   +-- index.html           # Landing page (Create/Join)  
|   +-- room.html            # Room page (Video + Chat + Queue)  
|   +-- css/style.css  
|   +-- js/  
|       +-- main.js  
|       +-- room.js  
+-- tests/                   # 13 test suites (85%+ coverage)  
+-- package.json  
+-- .eslintrc.json  
+-- .prettierrc.json  
+-- .env.example
```

Listing 2: Project Directory Structure